

# Data Communication

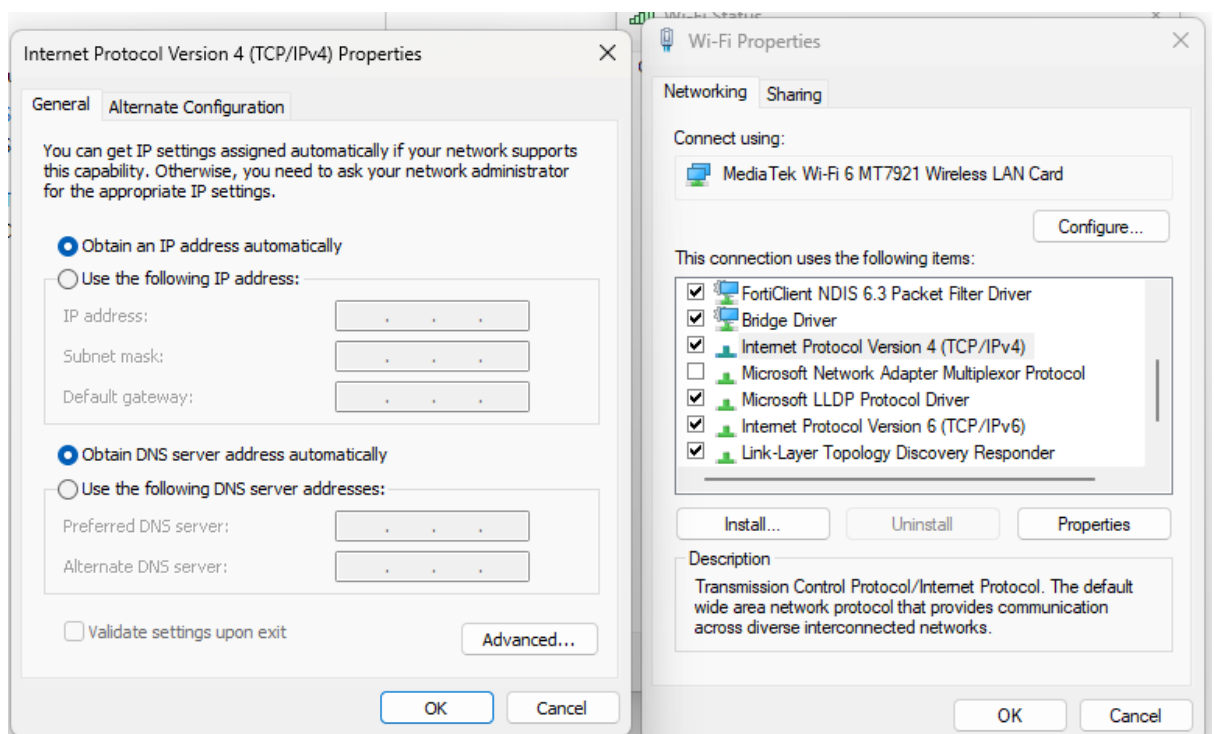
## Important port number

| Port #                   | Protocol                                        |
|--------------------------|-------------------------------------------------|
| 22                       | Secure shell                                    |
| 25                       | Simple Mail Transfer Protocol [SMTP] aka. Email |
| 53                       | Domain Name System [DNS]                        |
| 67:68 – [server: client] | Dynamic Host Configuration Protocol [DHCP]      |
| 80                       | Hypertext Transfer Protocol [HTTP]              |
| 143                      | Internet message Access Protocol [IMAP]         |
| 443                      | HTTP Secure [HTTPS]                             |

HTTP เมื่อก่อนจะใช้ TCP จนถึง HTTP/3: QUIC เป็นการใช้ UDP ใน application layer กับ TCP ใน transport layer จนกลายเป็น Standard ของ HTTP/3

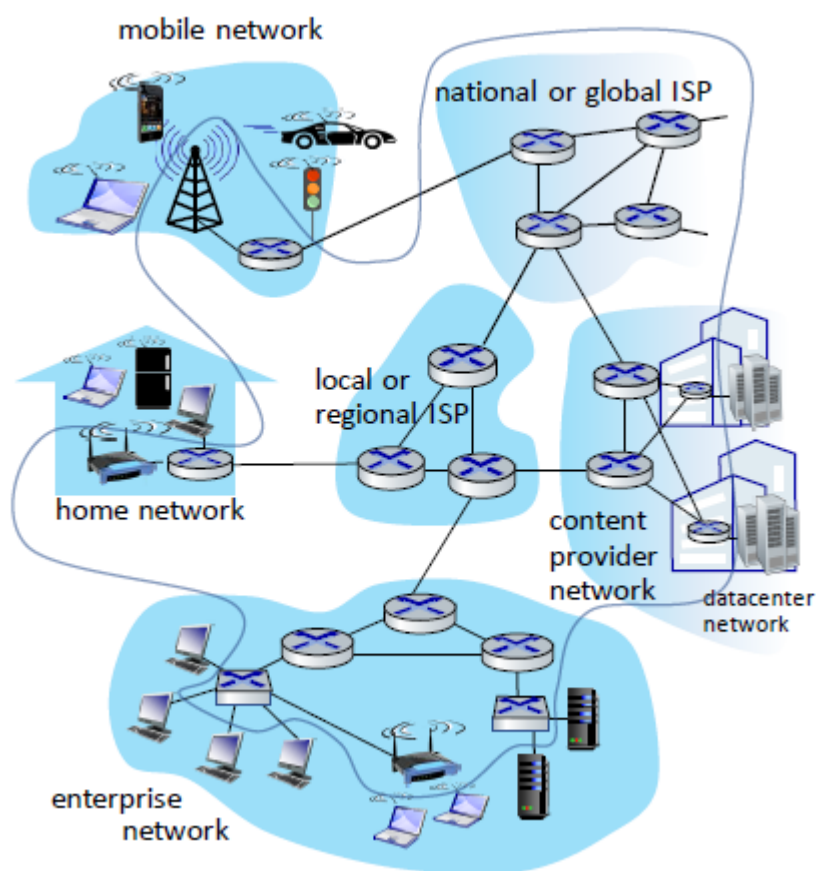
## การเชื่อมต่อ Internet [success]

- ต้องมีการ config: DNS, IP, subnet mask, Gateway



- Network address translator: ตัว mapping public IP -> private IP
- Router [network layer]
  - connect between network or router
  - Facilitate data traffic
- Switch [data link layer]

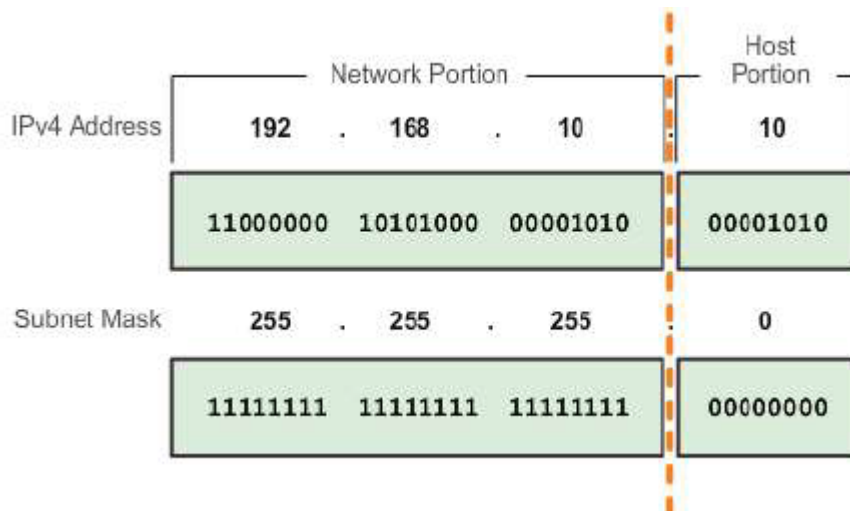
- connect devices in local network
  - assigning routing point
- Note router คือ ปลั๊กบ้าน switch คือ ปลั๊กพ่วง
- Network edge
    - Hosts: clients and servers
  - Access networks, physical media
    - wired
    - wireless
  - Network core
    - Interconnected routers
    - Network of networks



- Cable-based access:
  - Frequency division multiplexing [FDM]

## IP Addressing [IPv4] [success]

- ใช้ระบบเลขฐานสอง 32-bit
- แปลงจากฐานสิบเป็นฐานสอง
  - แปลงกลับ -> แบ่งฐานสองออกทีละ 8-bit
- Subnet mask: จุดที่ใช้บอกว่าส่วนไหนใน Address ที่เป็น host portion [อยู่ด้านหลัง / ของ network]



- ตัวอย่าง

|                                        | Dotted Decimal | Significant bits shown in binary |
|----------------------------------------|----------------|----------------------------------|
| Network Address                        | 10.1.1.0/24    | 10.1.1.00000000                  |
| First Host Address                     | 10.1.1.1       | 10.1.1.00000001                  |
| Last Host Address                      | 10.1.1.254     | 10.1.1.11111110                  |
| Broadcast Address                      | 10.1.1.255     | 10.1.1.11111111                  |
| Number of hosts: $2^8 - 2 = 254$ hosts |                |                                  |
| Network Address                        | 10.1.1.0/25    | 10.1.1.00000000                  |
| First Host Address                     | 10.1.1.1       | 10.1.1.00000001                  |
| Last Host Address                      | 10.1.1.126     | 10.1.1.01111110                  |
| Broadcast Address                      | 10.1.1.127     | 10.1.1.01111111                  |
| Number of hosts: $2^7 - 2 = 126$ hosts |                |                                  |
| Network Address                        | 10.1.1.0/26    | 10.1.1.00000000                  |
| First Host Address                     | 10.1.1.1       | 10.1.1.00000001                  |
| Last Host Address                      | 10.1.1.62      | 10.1.1.00111110                  |
| Broadcast Address                      | 10.1.1.63      | 10.1.1.00111111                  |
| Number of hosts: $2^6 - 2 = 62$ hosts  |                |                                  |

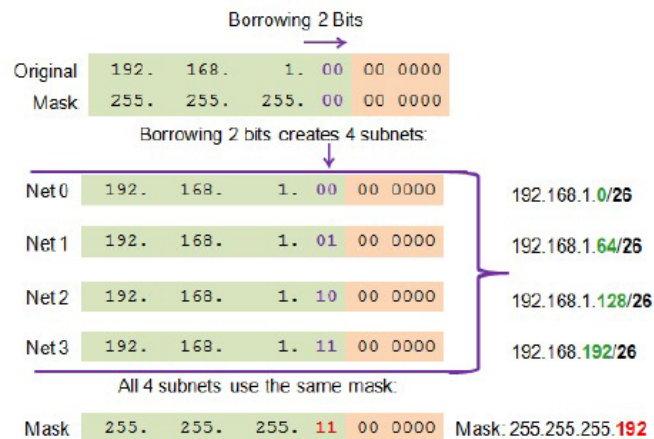
Note: Host แรกสุด(0) คือ Network address, Host ตัวสุดท้าย (ทุกบิต = 1) คือ Broadcast address

network แรกที่สามารถใช้ได้อยู่ที่ 1 และ network สุดท้ายอยู่ที่  $2^{\text{จำนวนบิตของ host}} - 1$

จำนวน host ที่ใช้ได้คือ  $2^{\text{จำนวนบิตของ host}} - 2$

- การหา Network Address
  - นำ Address ตั้งต้น AND กับ Subnet mask
  - Trick: ถ้า subnet mask = 255 จะ AND ออกมาได้ Address เดิม
- การสร้าง Subnets
  - ใช้แนวคิดของการยืม bit จาก network ที่มี

- Borrowing 2 bits to create 4 subnets.  $2^2 = 4$  subnets



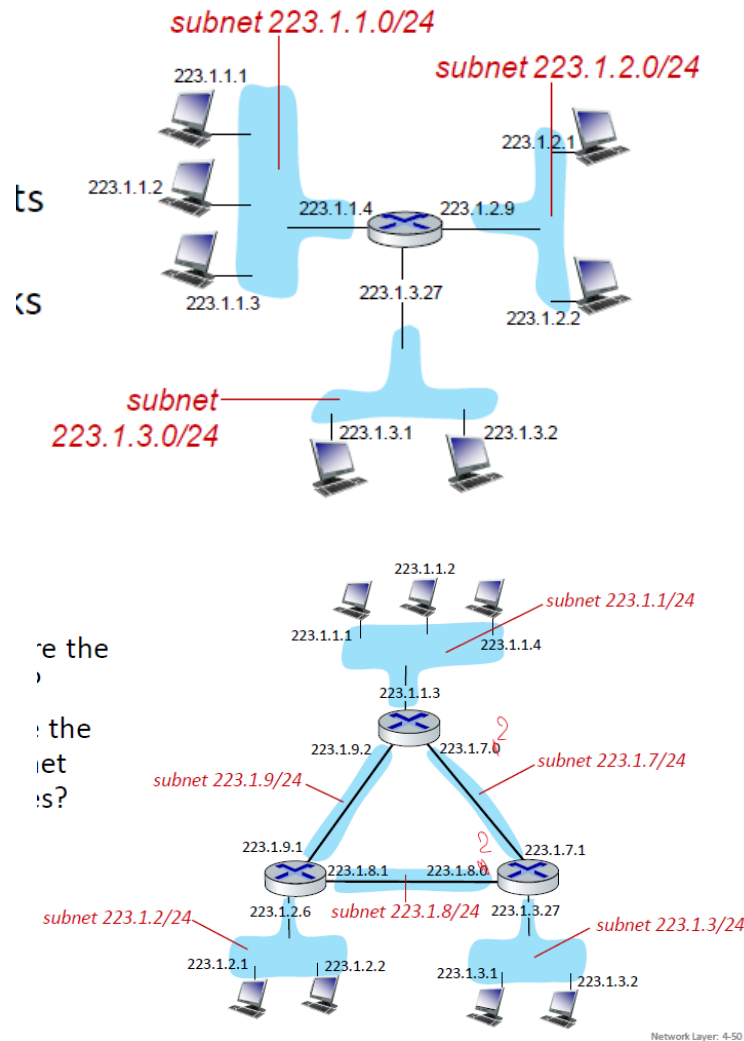
○

- Borrowing 3 bits to Create 8 Subnets.  $2^3 = 8$  subnets

|       |           |      |      |    |     |        |               |
|-------|-----------|------|------|----|-----|--------|---------------|
| Net 0 | Network   | 192. | 168. | 1. | 000 | 0 0000 | 192.168.1.1   |
|       | Fist      | 192. | 168. | 1. | 000 | 0 0001 | 192.168.1.1   |
|       | Last      | 192. | 168. | 1. | 000 | 1 1110 | 192.168.1.30  |
|       | Broadcast | 192. | 168. | 1. | 000 | 1 1111 | 192.168.1.31  |
| Net 1 | Network   | 192. | 168. | 1. | 001 | 0 0000 | 192.168.1.32  |
|       | Fist      | 192. | 168. | 1. | 001 | 0 0001 | 192.168.1.33  |
|       | Last      | 192. | 168. | 1. | 001 | 1 1110 | 192.168.1.62  |
|       | Broadcast | 192. | 168. | 1. | 001 | 1 1111 | 192.168.1.63  |
| Net 2 | Network   | 192. | 168. | 1. | 010 | 0 0000 | 192.168.1.64  |
|       | Fist      | 192. | 168. | 1. | 010 | 0 0001 | 192.168.1.65  |
|       | Last      | 192. | 168. | 1. | 010 | 1 1110 | 192.168.1.94  |
|       | Broadcast | 192. | 168. | 1. | 010 | 1 1111 | 192.168.1.95  |
| Net 3 | Network   | 192. | 168. | 1. | 010 | 0 0000 | 192.168.1.96  |
|       | Fist      | 192. | 168. | 1. | 010 | 0 0001 | 192.168.1.97  |
|       | Last      | 192. | 168. | 1. | 010 | 1 1110 | 192.168.1.126 |
|       | Broadcast | 192. | 168. | 1. | 010 | 1 1111 | 192.168.1.127 |
| Net 4 | Network   | 192. | 168. | 1. | 100 | 0 0000 | 192.168.1.128 |
|       | Fist      | 192. | 168. | 1. | 100 | 0 0001 | 192.168.1.129 |
|       | Last      | 192. | 168. | 1. | 100 | 1 1110 | 192.168.1.158 |
|       | Broadcast | 192. | 168. | 1. | 100 | 1 1111 | 192.168.1.159 |
| Net 5 | Network   | 192. | 168. | 1. | 101 | 0 0000 | 192.168.1.160 |
|       | Fist      | 192. | 168. | 1. | 101 | 0 0001 | 192.168.1.161 |
|       | Last      | 192. | 168. | 1. | 101 | 1 1110 | 192.168.1.190 |
|       | Broadcast | 192. | 168. | 1. | 101 | 1 1111 | 192.168.1.191 |
| Net 6 | Network   | 192. | 168. | 1. | 110 | 0 0000 | 192.168.1.192 |
|       | Fist      | 192. | 168. | 1. | 110 | 0 0001 | 192.168.1.193 |
|       | Last      | 192. | 168. | 1. | 110 | 1 1110 | 192.168.1.222 |
|       | Broadcast | 192. | 168. | 1. | 110 | 1 1111 | 192.168.1.223 |
| Net 7 | Network   | 192. | 168. | 1. | 111 | 0 0000 | 192.168.1.224 |
|       | Fist      | 192. | 168. | 1. | 111 | 0 0001 | 192.168.1.225 |
|       | Last      | 192. | 168. | 1. | 111 | 1 1110 | 192.168.1.254 |
|       | Broadcast | 192. | 168. | 1. | 111 | 1 1111 | 192.168.1.255 |

○

- Example: /24 subnets addresses



## Remark

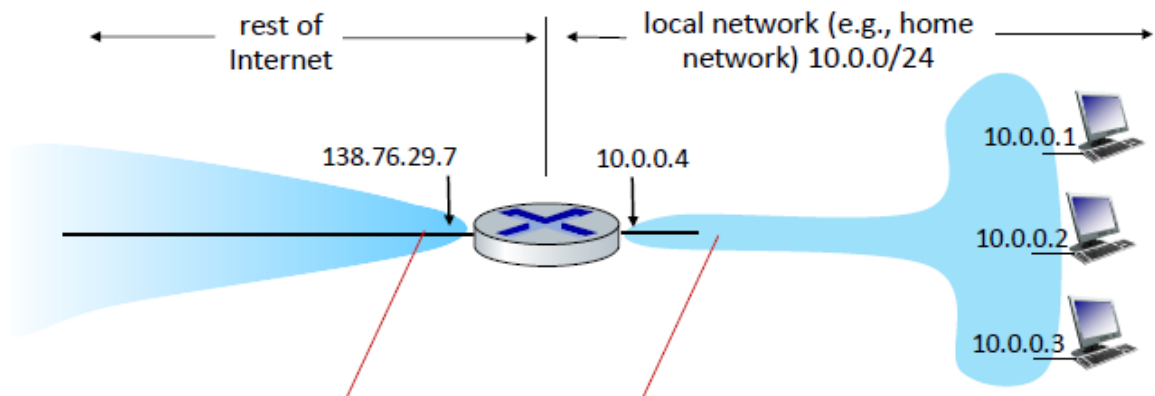
### Internet Corporation for Assigned Name and Numbers [ICANN]

- Allocates IPv4 address, through 5 regional registries
- Manage DNS root zone, including .com, .edu
- 32-bit not enough for IP -> NAT come to helps IPv4 address space exhaustion
  - IPv6 has 128-bit address space

### Network address translation [NAT]

- All devices in local network share just one IPv4 address as fas as outside world is concerned
- All datagrams leaving local network -> have same source NAT IP, different source port#

- Datagrams with source or destination in local have A.B.C/subnet address



#### Advantages:

- One IP needed from provider ISP for all devices
- Can change address of host in local without notifying outside world
- Can change ISP without changing address of devices in local
- Security -> devices in local not directly addressable, visible by outside

## Application layer [success]

เป็น distribute application ซึ่งรันบน end host/system

Ex. DNS, HTTP, HTTPS, SMTP, IMAP, CDNs, video streaming systems

Socket API: โปรแกรมบน LAN ใช้ได้ port

## Architecture

|                 |                                                                                                                           |                                            |
|-----------------|---------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| Client-server   | Server                                                                                                                    | client                                     |
|                 | Always-on host                                                                                                            | Contact, communicate with server           |
|                 | Permanent IP address                                                                                                      | May be intermittently connected            |
|                 | Often in data centers, for scaling                                                                                        | Dynamic IP address                         |
|                 |                                                                                                                           | Don't communicate directly with each other |
| Peer-peer (P2P) | No always-on server                                                                                                       |                                            |
|                 | End systems directly communicate                                                                                          |                                            |
|                 | Peers request service from other peers<br>: self-scalability – new peers bring new service as well as new service demands |                                            |
|                 | Change IP address<br>: Complete management                                                                                |                                            |

Processes communicating

- Same host, two processes communication using **inter-process** communication (defined by OS)
- Different hosts, communicate by **exchanging messages**

## Sockets

- Process sends/receives messages to/from sockets
- Socket is similar to door
  - Sending process shoves message out door
  - Sending process relies on transport infrastructure on other side
  - Two sockets involved: one on each side

## Protocol defines

1. Type of message exchanged e.g., request, response
2. message syntax: what fields in messages and how fields are describe
3. message semantics: meaning of information in field
4. rules: when and how process send & response

## ประเภท Protocol

1. open protocols: ทุก protocol บนเน็ตรู้จัก ถูกกำหนดโดย RFC (Request of comments) สามารถใช้งานร่วมกันได้ Ex. HTTP, SMTP

2. Proprietary protocols: โปรโตคอลเฉพาะ Ex. Skype, Zoom

## Transport service in app

1. data integrity: ยอมให้ข้อมูลหายหรือเปลี่ยนแปลงได้ไหม เช่น ถ้าเป็นการย้าย file จะยอมไม่ได้ แต่ audio ยอมได้
2. Timing: บางแอปต้องการดีเลย์ที่ต่ำ เช่น เกม
3. throughput: บางแอปมีการกำหนดขั้นต่ำไว้ บางแอปจะเปลี่ยนไปตามที่ใช้ เช่น
4. security: การเข้ารหัส

## Transport Service Requirements: common apps

|                        | <u>application</u> | <u>data loss</u> | <u>throughput</u>                        | <u>time sensitive?</u> |
|------------------------|--------------------|------------------|------------------------------------------|------------------------|
| file transfer/download |                    | no loss          | elastic                                  | no                     |
|                        | e-mail             | no loss          | elastic                                  | no                     |
|                        | Web documents      | no loss          | elastic                                  | no                     |
| real-time audio/video  |                    | loss-tolerant    | audio: 5Kbps-1Mbps<br>video:10Kbps-5Mbps | yes, 10's msec         |
| streaming audio/video  |                    | loss-tolerant    | same as above                            | yes, few secs          |
| interactive games      |                    | loss-tolerant    | Kbps+                                    | yes, 10's msec         |
| text messaging         |                    | no loss          | elastic                                  | yes and no             |

Note อะไรก็ตามที่ต้องการความถูกต้อง เช่น ไฟล์ ข้อความ ต้อง ไม่มีการสูญหายของข้อมูล และ ไม่มี throughput

### Protocol Services

#### TCP service

- Reliable transport
- Flow control: sender won't overwhelm receiver
- Congestion control: throttle sender when network overloaded
- Connection-oriented: required set up processes between client and server
- Not provides timing, minimum throughput guarantee, security

#### Securing TCP

- Provides encrypted TCP
- Data integrity
- End-point authentication
- How it implements in application layer
  - Apps use TLS libraries
  - Cleartext sent into "socket" encrypted

#### UDP service

- Unreliable data transfer
- Not provides reliability, flow control. Congestion control, timing, throughput guarantee, security or connection setup

## Internet application with transport protocols

| application            | application layer protocol                     | transport protocol |
|------------------------|------------------------------------------------|--------------------|
| file transfer/download | FTP [RFC 959]                                  | TCP                |
| e-mail                 | SMTP [RFC 5321]                                | TCP                |
| Web documents          | HTTP [RFC 7230, 9110]                          | TCP                |
| Internet telephony     | SIP [RFC 3261], RTP [RFC 3550], or proprietary | TCP or UDP         |
| streaming audio/video  | HTTP [RFC 7230], DASH                          | TCP                |
| interactive games      | WOW, FPS (proprietary)                         | UDP or TCP         |

## DHCP (Dynamic Host Configuration Protocol)

*host dynamically obtains IP address from network server when it joins network*

- can renew on address in use
- allows reuse of address
- support for mobile users who join/leave network

### Step

Client Discover network to Server.

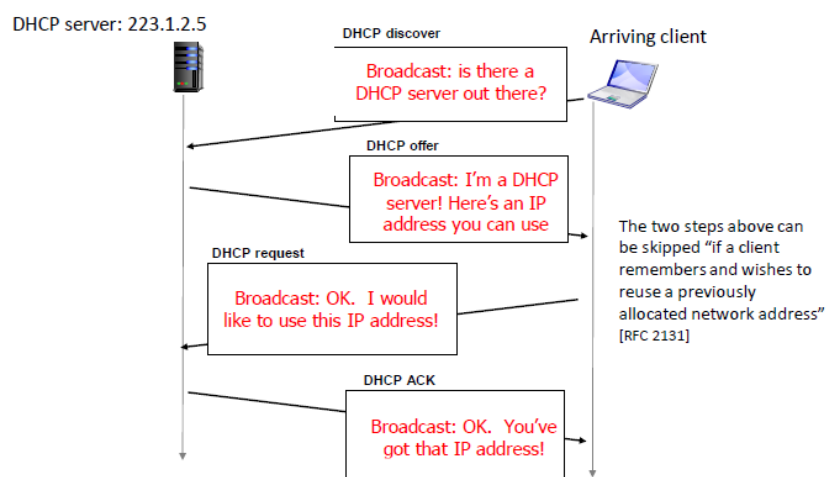
Server Offer IP to Client.

Client request IP to Server.

Server Acknowledge IP to Client.

Note: DHCP not just IP addresses

- Address of first-hop router for client
- Name and IP address of DNS server
- Network mask



## HTTP (Hypertext transport protocol)

Web browser(client) <-> internet <-> web servers

### Model

Client: browser -> requests, receives and display web objects

Server: server sends response

## Process

- Client initiates TCP connection on port 80
- Server accepts from client
- HTTP messages exchanged between browser and web server
- TCP connection closed

Note: HTTP is stateless = no information about past client requests

Note: protocol that maintain “state” are complex

## Type of connections

### 1. Non-persistent HTTP

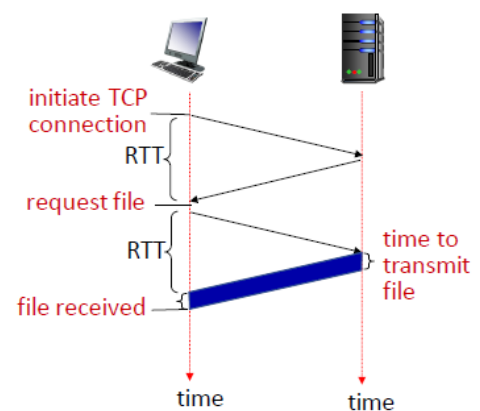
- one object sent. Download multiple objects required multiple connections
- response time =  $2RTT + \text{file transmission time}$
- one RTT to initiate TCP connection
- one RTT for request and first few bytes response to return

Note: RTT means time that client sent packet to server and back

**Problem** -> 2RTTs per object -> OS overhead, waste bandwidth

### 2. Persistent HTTP

: multiple objects can be sent over single TCP connection



## Request messages

### Component

- request line [GET, POST, HEAD commands]
- header line [Domain name, User-agent, Accept, Accept-Language, Accept-Encoding, Connection]
- carriage return.

# HTTP request message

- two types of HTTP messages: *request, response*

- HTTP request message:**

- ASCII (human-readable format)

request line (GET, POST, HEAD commands) → GET /index.html HTTP/1.1\r\n

header lines → Host: www-net.cs.umass.edu\r\n  
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:80.0) Gecko/20100101 Firefox/80.0 \r\n  
Accept: text/html,application/xhtml+xml\r\n  
Accept-Language: en-us,en;q=0.5\r\n  
Accept-Encoding: gzip,deflate\r\n  
Connection: keep-alive\r\n  
\r\n

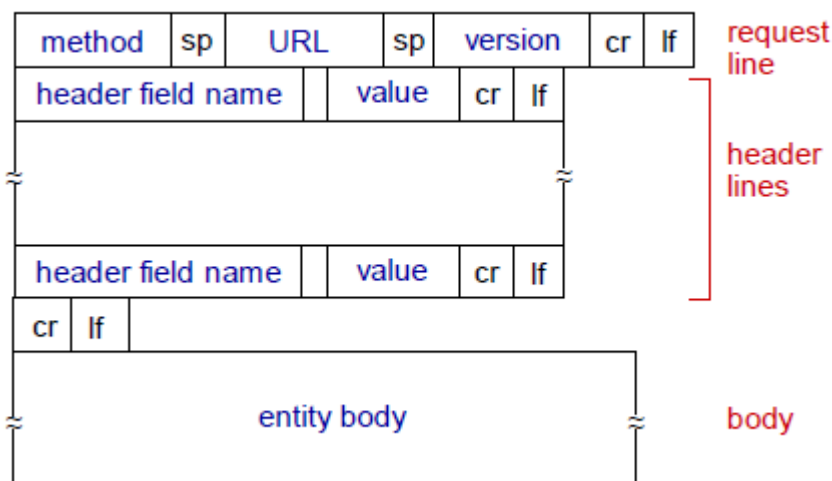
carriage return character  
line-feed character → \r\n

carriage return, line feed at start of line indicates end of header lines → \r\n

\* Check out the online interactive exercises for more examples: [http://gaia.cs.umass.edu/kurose\\_ross/interactive/](http://gaia.cs.umass.edu/kurose_ross/interactive/)

Application Layer: 2-26

## General format



## Other request messages

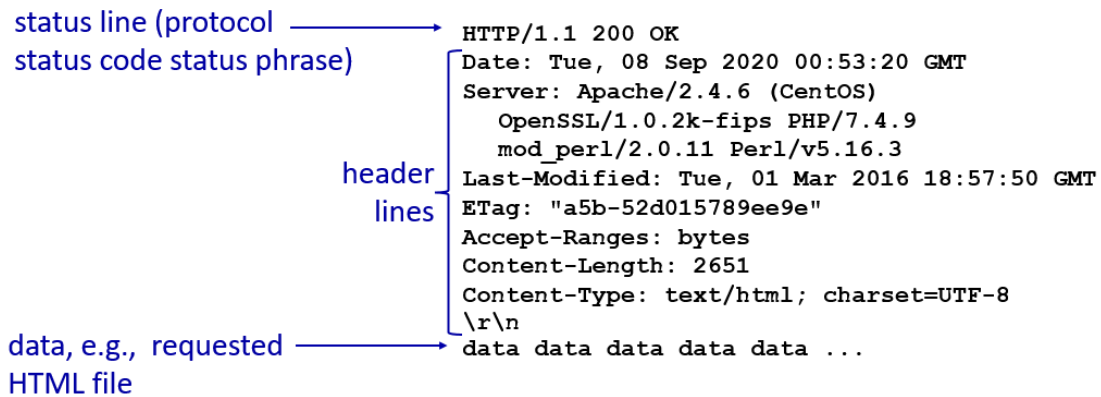
- POST: includes form input, user input sent from client to server in entity body.
- GET: sending data to server, include user data in URL field.
- HEAD: request headers that would be returned if specified URL with an HTTP GET method.
- PUT: uploads new object to server, completely replaces file that exists at specified URL with content in entity body

## Response message

### Component

- status line
- header lines
- data, e.g. requested HTML file.

# HTTP response message



## Status codes

200 OK: request succeeded

301 Moved Permanently: requested object moved, new location later this.

400 Bad Request: request msg not understood by server

404 Not Found: requested document not found

505 HTTP Version Not Supported

## Cookies

- Between web sites and client's browser

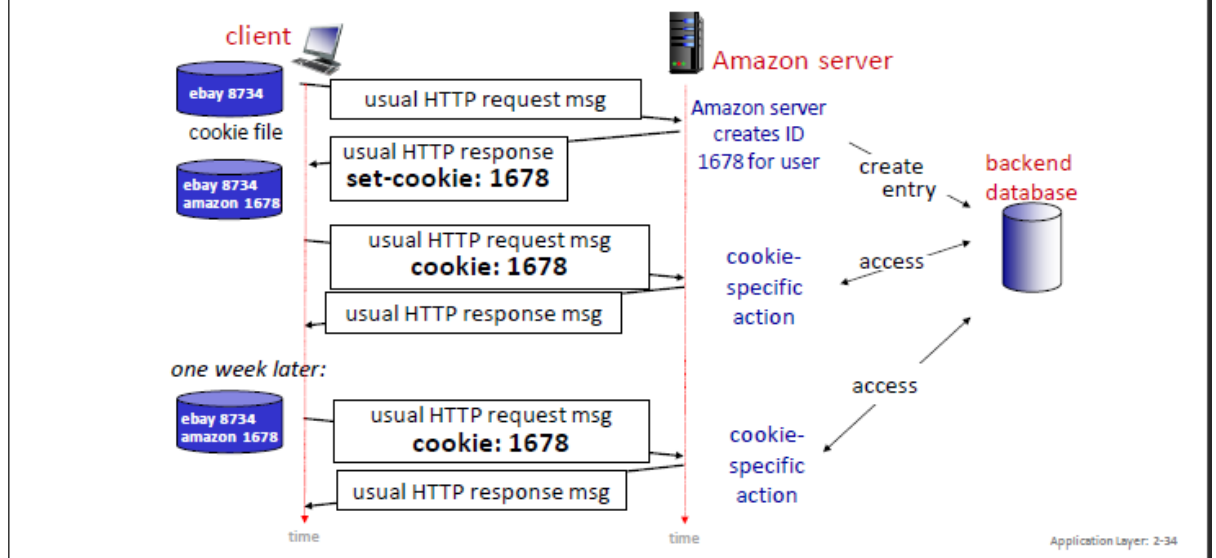
## Components

- Cookie header line of HTTP response message
- Cookie header line in next HTTP request message
- On user's host manages by user's browser
- Back-end database website

## Step

- Client sent request msg
- Server creates cookie id -> create entry to backend database, send response set-cookie to client
- Using -> client request msg with cookie id -> server get cookie and call access to database.

## Maintaining user/server state: cookies



- Used for
  - Authorization
  - Shopping carts
  - Recommendations
  - User session state (Web e-mail)

## E-mail [SMTP, IMAP]

### Components

- User agents
  - A.K.A mail reader
  - Composing, editing, reading mail messages
    - Outlook, iPhone mail client
  - Incoming messages stored on server
- Mail servers
  - Mailbox contains incoming messages for user
  - Message queue of outgoing mail messages

### Simple mail transfer protocol [SMTP]

- SMTP uses TCP port 25 to transfer email message from client to server
- Three phases of transfer
  - SMTP handshaking
  - SMTP transfer of messages
  - SMTP closure
- Command/response interaction
  - Commands: ASCII text
  - Response: status code and phrase
- Client: sending mail server

- Server: receiving mail server

Note: เมลล์จะถูกเก็บไว้ใน server จนกว่าจะมีการเปิดเมลล์ใน agents

## Internet Mail Access Protocol [IMAP]

- Retrieval, deletion, folders of stored messages on server

## DNS (Domain Name System)

### Services

- Translate hostname to ip
- Host aliasing
- Mail server aliasing
- Load distribution: many IP correspond to one name

### Q: Why not centralize DNS -> doesn't scale

- Single point of failure
- Traffic volume
- Distant centralized database
- Maintenance

### Things about DNS

- Humongous distributed database
  - Billion records
- Handles many trillions of queries/day
  - Many reads than writes
  - Performance matters: almost internet transaction interacts to DNS
- Organizationally, physically decentralized
  - Millions of different organizations responsible for their records
- Bulletproof -> reliability, security

## Distributed, hierarchical database

### Root

- 13 logical root
- No security -> DNSSEC provides security (authentication, message integrity)
- ICANN manage root DNS domain

### Top level Domain

- e.g., .com, .org, .edu include contry domain

### Authoritative

- organization's own DNS server, hostname to IP mapping
- maintained by organization or service provider

- e.g., Yahoo.com, umass.edu

## Resolution

### *Iterated query*

- contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”
- host <-> local <-> ask root, TLD, authoritative

### *Recursive query*

- heavy load at upper levels
- host <-> local <-> root <-> TLD <-> authoritative

## Caching information

- once name server has mapping
  - improve response time
  - entries timeout after some time (TTL)
  - TLD server cached in local name servers
- Cached maybe out-of-date
  - If named host change IP, Internet not know until all TTLs expire
  - Best effort name to address translation

## DNS records

- RR record [Resource record] -> format:(name, value, type, ttl)

### ○ Type A

- Name is hostname
- Value is IP address
- domain.com , 192.33.22.1

### ○ Type NS

- Name is domain
- Value is host of authoritative name
- facebook.com, b.facebook.com

### ○ Type CNAME

- Name is alias
- Value is canonical name
- [www.facebook.com](http://www.facebook.com), canonical name = star-mini.c10r.facebook.com

- Type MX

- value is SMTP mail server

## How to get your info into DNS

- Register name at DNS registrar with
  - Name
  - Ip address of authoritative name server [pre-config]
  - Registrar inserts NS, A, RRs into .com TLD server

## DNS security

### DDoS attacks

- bombard root server with traffic
- bombard TLD servers: potentially more dangerous

### Spoofing attacks

- redirecting users to fake sites

## Video Streaming and CDN

- streaming video traffic is major consumer of internet bandwidth
- Challenge: scale
- Challenge: heterogeneity
  - Different users have different capabilities
- Solution
  - Distributed, application-level infrastructure

## Multimedia: video

- CBR (constant bit rate)
  - encoding rate fixed
- VBR (variable bit rate)
  - Encoding rate changes as amount of spatial, temporal coding changes
  - Depends on bandwidth

## Streaming: Dynamic adaptive streaming over HTTP [DASH]

- Main challenges
  - Server-to-client bandwidth over time, changing network
  - Packet loss, delay, poor video quality
- Other challenges
  - Client interactivity: pause, fast-forward, rewind, jump through
  - Video packets may be lost, retransmitted

## Server

- Divides video file into multiple chunks

- Each chunk encoded at multiple different rates
- Different rate encoding stored in different files
- File replicate in various CDN nodes
- **Manifest file** -> provide URLs for different chunks

### Client

- Estimates server to client bandwidth
- Request one chunk at a time
  - Choose coding rate sustainable with current bandwidth
  - Can chose different coding rate at different points, and from different servers
- INTELLIGENCE at client
  - When to request chunk
  - What encoding rate to request
  - Where to request chunk

Note: Streaming video = encoding + DASH + playout buffering

## Transport layer

Communicate between processes

Logical communication: action in end systems.

**Sender:** break msg into segment | passes to network layer -> determines segment header -> creates segment -> passes segment to IP

**Receiver:** reassembles segment into msg | receives segment from IP -> checks header values -> extracts application-layer msg -> demultiplexes message up to application via socket

Not available: Delay guarantees, Bandwidth guarantees

### REMINING

Thread = each process executing in parallel and share memory and resources.

Fork = copy of the current process. Don't share memory and resources

## Multiplexing/Demultiplexing

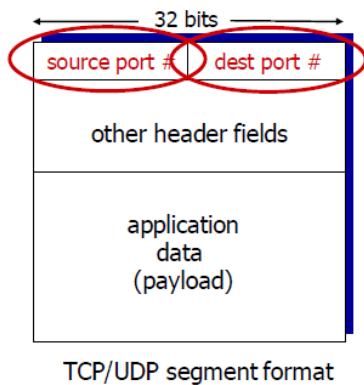
Network แยก thread process -> look up to source IP and source port to know where is the client

### Multiplexing (รวม)

As sender: handle data from multiple sockets, add transport header

### Demultiplexing (แยก)

As receiver: Use header info to deliver received segments



Note IP datagram = UDP

Port number exist  $2^{16} \rightarrow 0-65534$

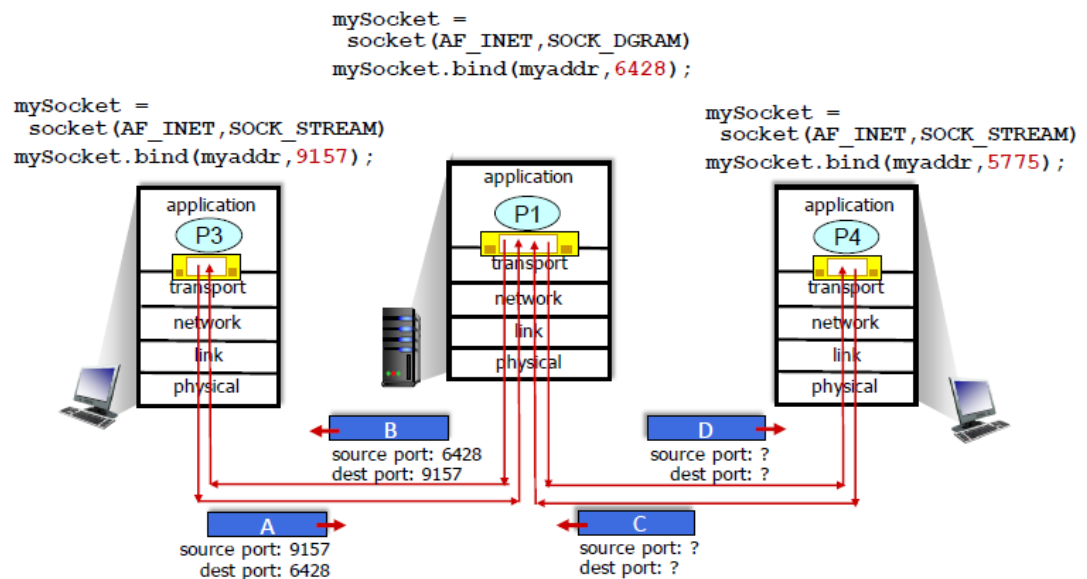
Host receives IP datagrams

- IP address, destination IP address
- 1 transport-layer segment
- source, destination port number

### Connectionless demultiplexing

- Create socket: specify host-local port number
- Create UDP socket: specify destination IP address and destination port #
- Host receives UDP segment:
  - Check destination port# in segment
  - Directs UDP segment to socket with that port#
  - UDP with same dest, port# but different source IP or source port  $\rightarrow$  directed to same socket

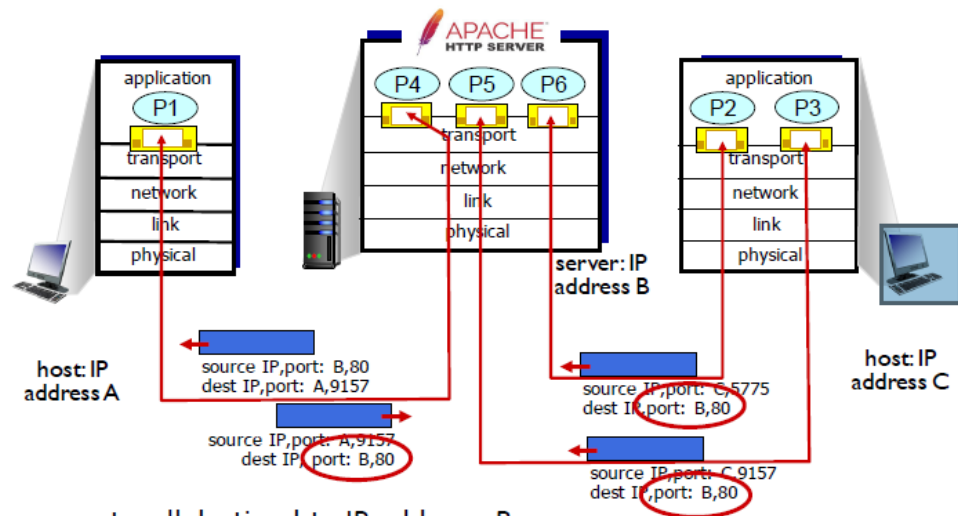
## Connectionless demultiplexing: an example



### Connection-oriented demultiplexing

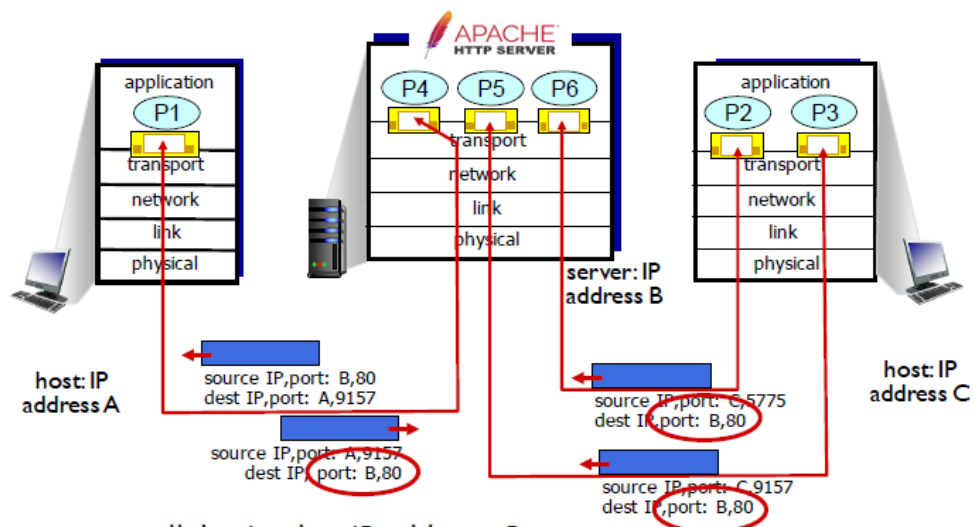
- TCP socket
  - Source IP address
  - Source port #
  - Destination IP address
  - Destination port #
- Server support many parallel TCP sockets: each socket handle different connecting client

## Connection-oriented demultiplexing: example



- Three segments, all destined to IP address: B

## Connection-oriented demultiplexing: example



- Three segments, all destined to IP address: B,  
dest port: 80 are demultiplexed to *different* sockets

### Summary

- multiplexing, demultiplexing: based on segment, datagram header field values
- UDP demultiplexing -> only port number
- TCP demultiplexing -> 4 tuples: source and destination IP and port
- Multi/demultiplexing happen **all** layers

### UDP (User Datagram Protocol /Connectionless transport)

- No connection between Sender and Receiver
- Unreliable, unordered delivery
- Best-effort IP -> may be lost, out of order
- Connectionless: no handshaking, UDP segment indecently of others

## Why is there a UDP

- No connection establishment
- No connection state
- Small header size
- No congestion controls

## Transport Layer Actions

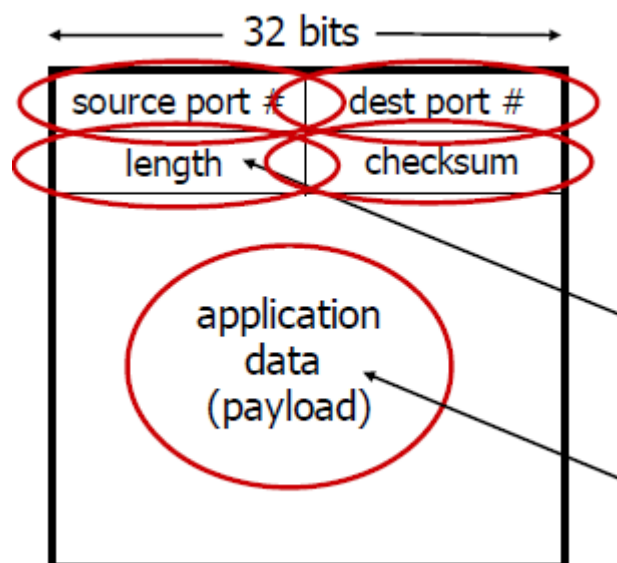
### Sender actions:

- Passed an application layer
- Determines UDP segment header fields values
- Creates UDP segment
- Passes segment to IP

### Receiver actions:

- Receives segment from IP
- Checks UDP checksum header value
- Extracts application-layer message
- Demultiplexes message up to application via socket

### UDP segment header



### Checksum

- Chop segment data (divide by 16)

example: add two 16-bit integers

|            |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|------------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
|            | 1 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 |
|            | 1 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| wraparound | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 |
| sum        | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| checksum   | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 |

-

## Principle of reliable data transfer

- Idea: checksum, respond msg, timeout, Sequence number, Acknowledgment number

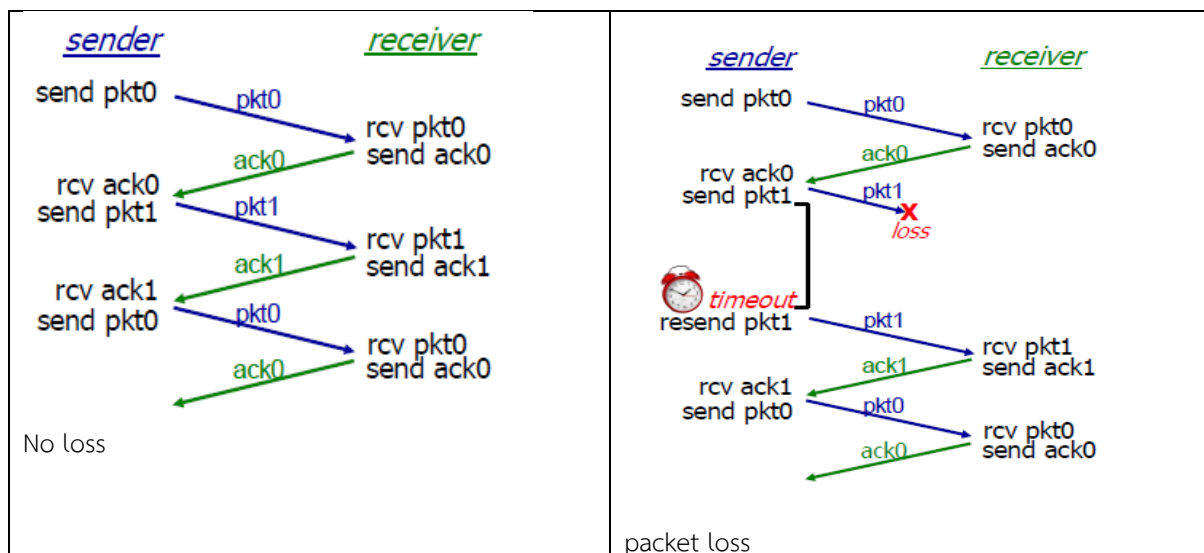
## rdt (Reliable data transfer protocol)

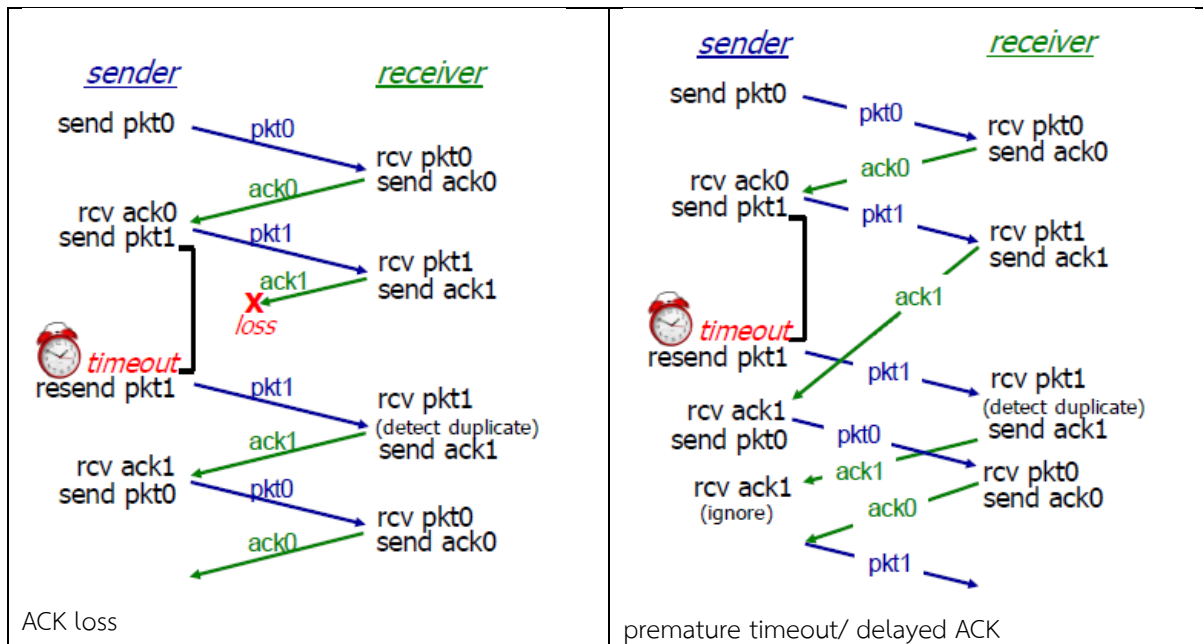
Reliable: no bit errors, no loss packets

### Rdt3.0 channels with errors and loss

- Channel can also lose packets (data, ACKs) -> checksum, sequence# are not enough
- Sender waits has timeout
  - If pkt (or ACK) just delayed
    - Retransmission will duplicate -> seq# handles this
    - Receiver specify seq# of packet being ACKed

### Rdt3.0 in action

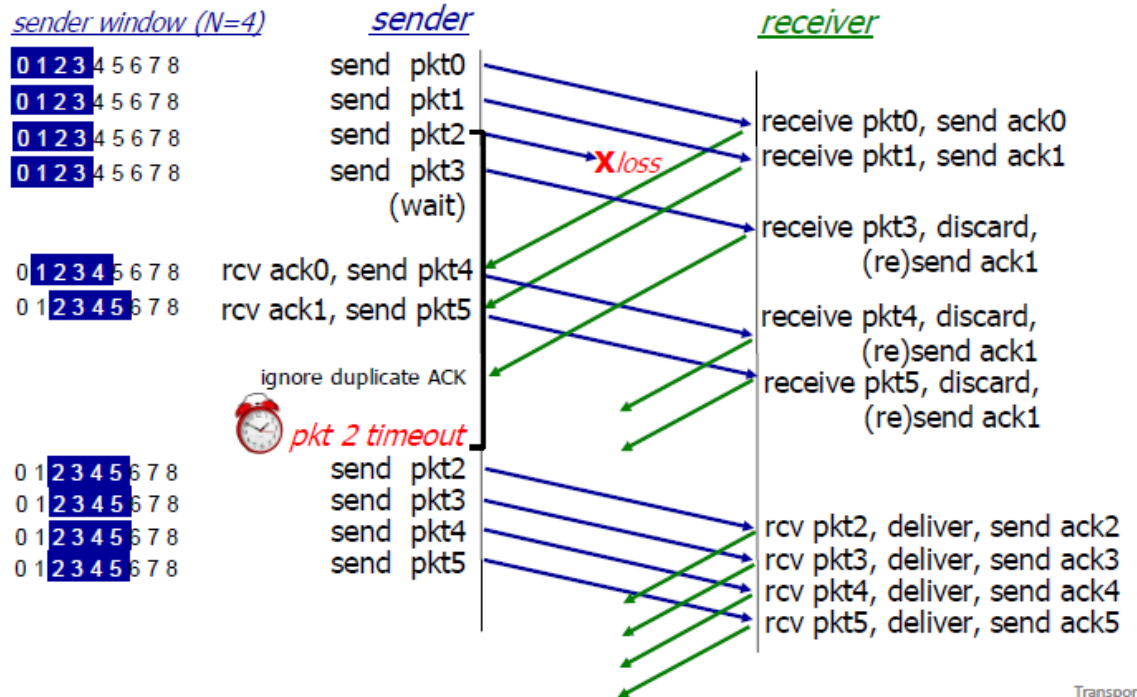




## Mechanism handling

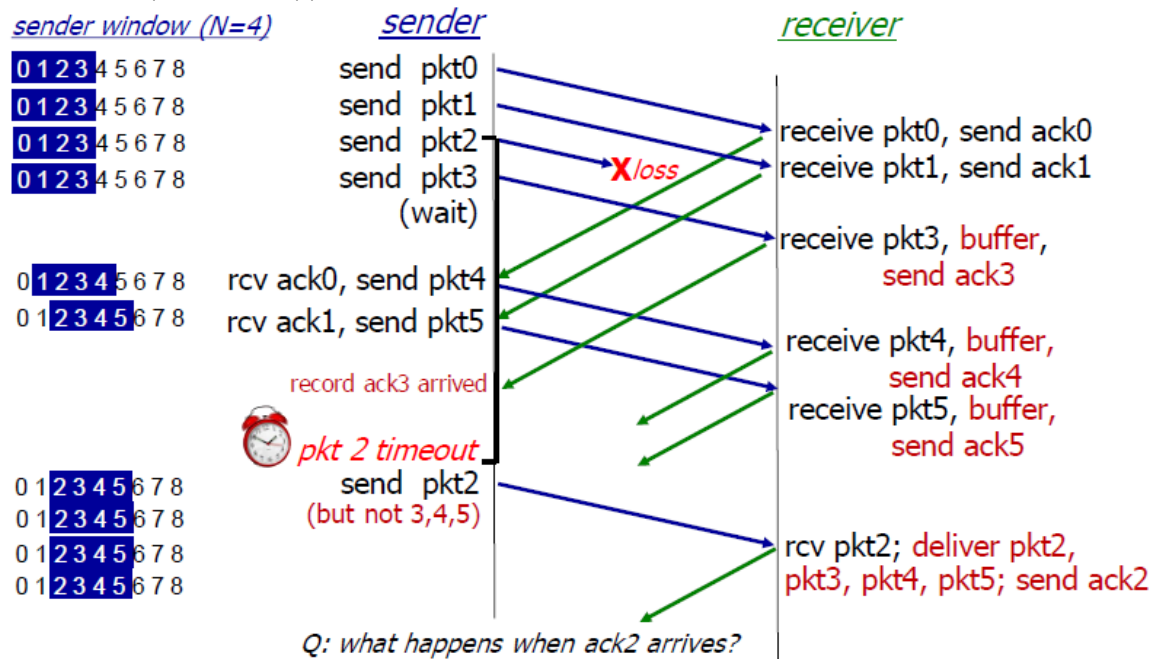
Go-Back-N: ส่งใหม่ทั้งหมดตั้งแต่อันที่หาย (N: get from application layer)

- Easy at receiver. No complicate
- Sender: send n package
  - New package sends after receive ACK from receiver
  - If receive dup ACK -> ignore dup ACK
  - If timeout -> resend all the package after latest receive ACK
- Receiver: receive package and send acknowledge
  - If receive the unexpected package -> discard package and (re)send latest acknowledge



*Selective repeat: ส่งใหม่เฉพาะอันที่หายไป*

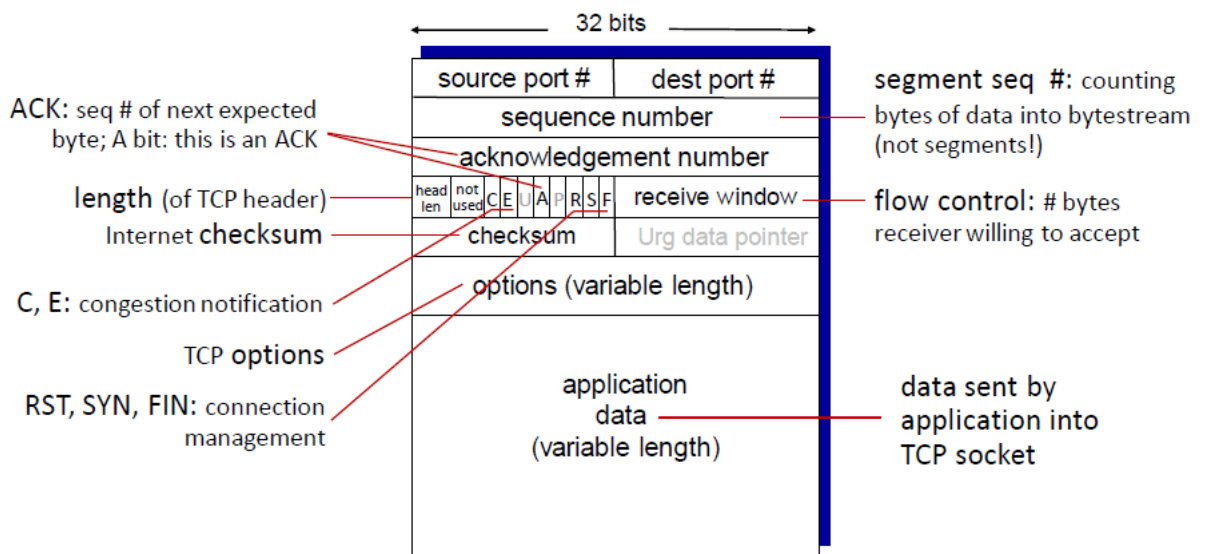
- Complicated in sender and receiver device.
- Flow control is needed: Sender window
- Pipelining: multiple packets in flight
- Buffer packets. Collect the packet that come after the expect packet.
- Saving bandwidth
- After receive -> deliver package to application layer [if it not in buffer]



## TCP (Transmission Control Protocol)

- Reliable, in-order delivery

- Congestion control: ป้องกันการสูญหายของข้อมูล
- Flow control: จัดการเรื่องความเร็วในการส่งให้รับส่งพอกัน
- Connection setup: Connection-oriented: Connection between Sender and Receiver
  - Overhead
- Point-to-point
- Cumulative ACKs: if ACKs loss but get ACK# after that loss, it not retransmits. [assume that receiver get all data]
- Full duplex data: can be sender and receiver in one
- Pipelining: has window size
- Connection-oriented: 3-way handshaking -> request, response, initialize state.



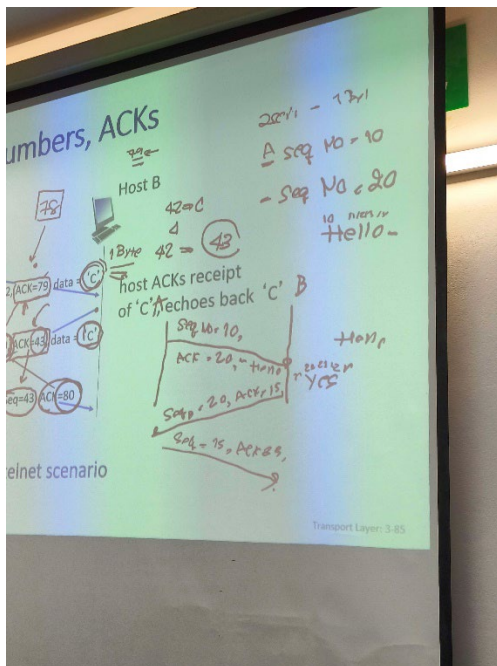
## TCP sequence numbers, ACKs

### Sequence number

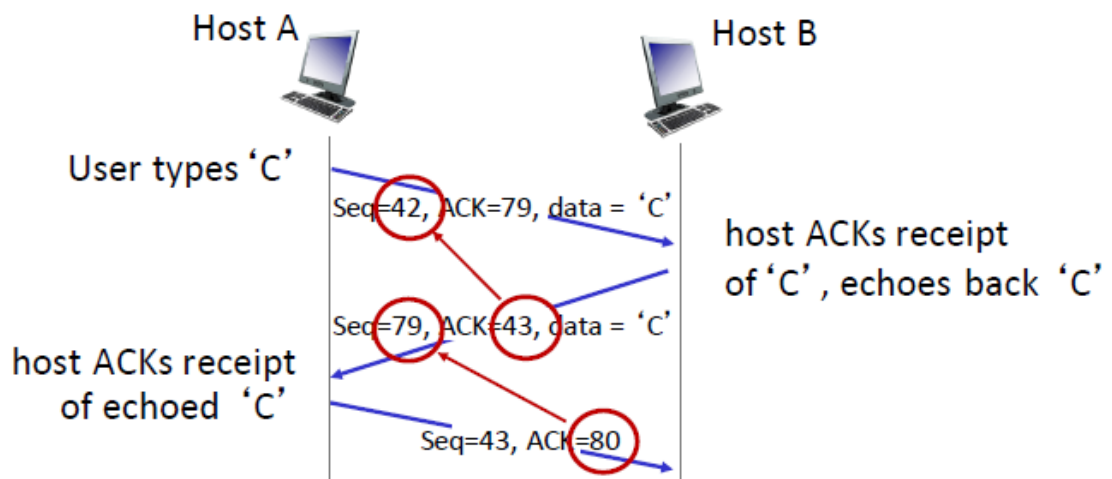
- can start with any number but have to be in sequence after

### Acknowledgement:

- Seq: กำลังจะส่งข้อมูล Byte ที่เท่าไร EX. 42 = byte ที่ 42
- Cumulative ACK: Seq+Byte ของข้อมูล



-



-

- Fast-transmit: after get 3 dup ACKs -> retransmit

### 3-way handshake

- Client communication to server
- Server acknowledges to client
- Client acknowledge to server